ALGORITMOS DE PROGRAMAÇÃO

Prof. Me. João Víctor Ramos

TRABALHO – MERCADO DIGITAL

TEMA DE ESTUDO

- Funções e procedimentos. Registros.

OBJETIVOS

- Compreender a aplicar os conceitos de modularização.
- Compreender e aplicar registros usando uma linguagem de programação.

DESCRIÇÃO DA ATIVIDADE

Neste trabalho a equipe deverá construir um programa que solucione uma necessidade de um varejista o qual tem a necessidade de migrar a venda de seus produtos para o computador. Neste primeiro momento deverá ser realizado um programa que possibilite:

- Cadastrar clientes
- Cadastrar produtos
- Listar clientes cadastrados
- Listar produtos cadastrados
- Efetuar uma venda simples
- Sair do programa

A equipe contará com o apoio de etapas no desenvolvimento do trabalho que deverá segui-las, ficando livre para implementar novas funcionalidades que não estiverem listadas desde que cumpram com os itens principais destas etapas.

ETAPAS

1) Os dados do programa a ser desenvolvido deverá ser mantido em memória, ou seja, em estruturas de variáveis, estruturas homogêneas (vetores) e heterogêneas (registros), sendo que os principais dados a serem mantidos foram organizados nos seguintes registros. **Você não pode mudar o tipo de dados de cada campo** o que é int deve manter int, o que é char deve manter char e assim por diante.

Clientes	
int	codigo
char	nome[50]
double	cpf
char	sexo
telefones	tel

Produtos	
int	codigo
char	nome[20]
modelos	modelo
double	valor

Carrinhos	
clientes	cliente
produtos	produto[3]
int	quantidade[3]
double	subtotal
double	desconto
double	total

Telefones	
char	fixo[15]
char	movel[16]

Modelos	
char	marca[20]
char	modelo[20]

2) A estrutura contém os dados básicos de cada conjunto de informação que armazena. Abaixo estão comentadas algumas regras que são mais relevantes:

   A) Na estrutura CLIENTES
      i. clientes.codigo = é formado pelos 6 primeiros dígitos do CPF do cliente.
      ii. clientes.tel = é uma estrutura TELEFONES que possui fixo e celular como campos.
   B) Na estrutura TELEFONES
      i. Os campos fixo e móvel são strings a fim de guardar, além do número, a máscara do telefone como no exemplo: (11) 3210-1000

C) Na estrutura PRODUTOS
   i. produtos.codigo = é formado pelo valor inteiro dos 2 primeiros caracteres da marca do produto e 2 caracteres do modelo do produto (tabela ASCII). Após isso aplicar a fórmula $hash=C1\times31^3+C2\times31^2+C3\times31^1+C4\times31^0$ onde C1, C2, C3 e C4 são os valores numéricos inteiros dos caracteres. Por exemplo: Marca: **DE**LL Modelo: **PR**ECISION iremos usar "DEPR". Sabendo que os valores inteiros das letras são: D → 68; E → 69; P → 80; R → 82 aplicamos na formula $hash= 68\times31^3+69\times31^2+80\times31^1+82\times31^0$ que resulta hash= 2093659. O código será então 2093659.
   ii. produtos.modelo = é uma estrutura MODELOS que possui marca e modelo como campos.

D) Na estrutura CARRINHOS
   i. carrinhos.cliente = é uma estrutura CLIENTES que guarda os dados do cliente para quem a venda está sendo feita
   ii. carrinhos.produto[3] = é um VETOR de estruturas PRODUTOS que permite guardar ao menos 3 dados de produtos que estão sendo vendidos.
   iii. carrinhos.quantidade[3] = é um VETOR de inteiros que irá guardar a quantidade de cada item adicionado ao carrinho. Há uma relação onde: carrinhos.quantidade[0] possui a quantidade de produtos referente ao carrinhos.produto[0], carrinhos.quantidade[1] possui a quantidade de produtos referente ao carrinhos.produto[1] e assim por diante.
   iv. carrinhos.subtotal = guarda a soma de todos os valores da venda. Lembrando que é a soma de qnt *valor produto.
   v. carrinhos.desconto = guarda o valor do desconto (exemplo 10 para 10%). Também pode ser modificado para validar um cupom de desconto caso a equipe queira fazer algo diferenciado.
   vi. carrinhos.total = guarda o valor total da compra subtraído o valor do campo desconto. Por exemplo, se R$ 1.000,00 é o subtotal da compra e o desconto é de 10%, o valor deste campo total é de R$ 900,00.

3) O programa consiste basicamente em manter um laço de repetição que irá mostrar um menu inicial onde nele o usuário irá fazer a escolha das opções que deseja operar. Quando o usuário escolhe pela opção 0 o sistema é encerrado e fechado.

4) Para que seja possível guardar as informações referentes a estrutura, fica limitado ao **mínimo** de 3 clientes e 10 produtos cadastrados. Valores maiores que este fica opcional para cada equipe desenvolver se quiser.
   A) Para que seja possível guardar a lista de clientes e de produtos, inicializar uma VETOR de CLIENTES e um VETOR de PRODUTOS, uma vez que estes conterão no mínimo 3 e 10 itens cada.
      i. Exemplo: **struct** produtos produto[10]; **struct** clientes cliente[3];
   B) Para o carrinho não há necessidade de guardar mais de uma venda neste primeiro momento, portanto pode-se inicializar apenas uma estrutura simples de carrinho. Fica opcional à equipe caso queira manter mais de um carrinho.
      i. Exemplo: **struct** carrinhos carrinho.

5) Validações ao longo do programa são importantes é recomendado que sejam feitas pela equipe, porém **somente após finalizar todo o programa**. Alguns exemplos de validações podem ser vistos abaixo:
   A) Validar limite de clientes e produtos cadastrados. (dentro dos limites suportados pelo sistema)
   B) Validar valores de quantidade e valor de produtos (maior que 0)
   C) O programa não pode vender mais unidades do que existem em estoque.
   D) Validar caso os códigos não sejam encontrados.
   E) Validar códigos duplicados (quando um código gerado para um produto já existe e tratar a situação sem sobrescrever um cadastro anterior)
   F) O carrinho permite até três tipos de produto, mas deve-se tratar corretamente a tentativa de adicionar um quarto tipo e a inclusão de mais unidades de um item já existente.
   G) etc.

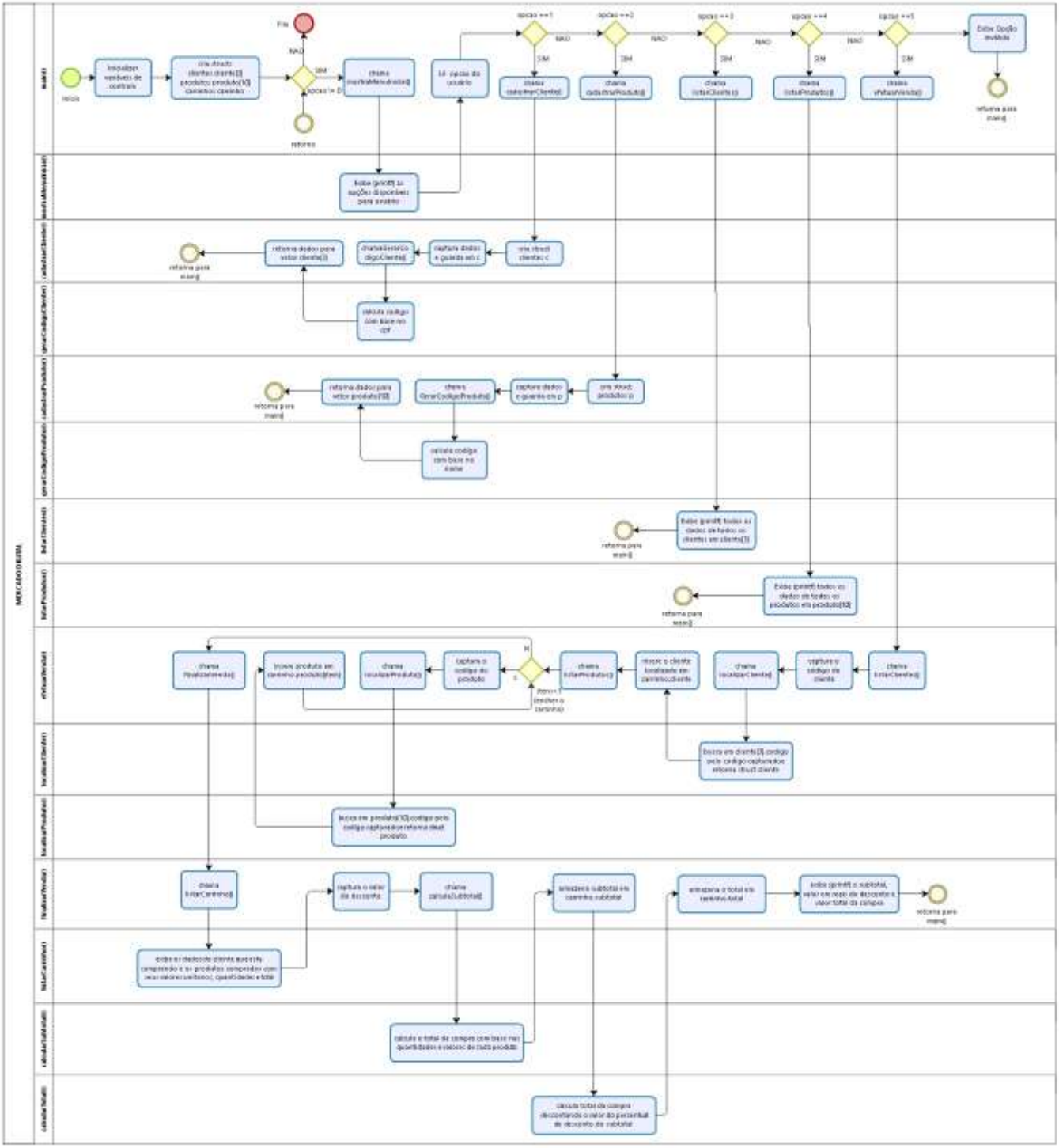
6) A programação **deverá ser feita de maneira modular**, ou seja, criando **funções e/ou procedimentos** para auxiliar a execução das tarefas. A equipe deverá se orientar na lista abaixo. Na lista exemplos que devem ser inseridos no programa. A equipe fica livre para criar as funções e/ou procedimentos que achar necessário para o desenvolvimento deste programa além das listadas abaixo.

Assinatura	Descrição
void main();	Função principal do programa. Nela deverá ser criado a struct para a lista de clientes, a lista de produtos e o carrinho. Nesta estrutura um laço controla a execução do programa de acordo com a seleção do menu de opções a ser apresentado. Quando o usuário desejar encerrar o programa ele sai do laço e encerra esta função.
void mostraMenuInicial();	Função sem retorno e sem passagem de parâmetros que exibe o menu inicial do programa contendo as opções que o cliente pode acessar: Cadastros, Listagens, Efetuar Compra e Sair do Programa. Deverá ser chamada na abertura do programa e sempre que sair de uma opção escolhida pelo cliente.
struct clientes cadastrarCliente();	Função sem passagem de parâmetros que retorna um struct com os dados do cliente. Deverá ser chamada quando for preciso cadastrar um cliente.
int geraCodigoCliente(**double** cpf);	Função que passa o CPF e retorna um inteiro com o código do cliente. Deverá ser chamada quando for preciso gerar o código do cliente. O código do cliente é formado pelos 6 primeiros dígitos do CPF. Exemplos: CPF 977.345.456-10 código 977345, CPF 032.123.456-00 código 32123.
void listarClientes(**struct** clientes cliente[3]);	Função sem retorno e que passa uma struct com os clientes cadastrados a fim de exibir na tela a lista de todos os clientes cadastrados. Deverá ser chamada quando solicitado no menu inicial e também ao iniciar uma venda para indicar quem está realizando a compra.
struct produtos cadastrarProduto();	Função sem passagem de parâmetros que retorna um struct com os dados do produto. Deverá ser chamada quando for preciso cadastrar um produto.
int geraCodigoProduto(**char** nomeProduto[30]);	Função que passa o nome do produto e retorna um inteiro com o código do produto. Deverá ser chamada quando for preciso gerar o código do produto. O código do produto é formado pela formula *hash* apliacada ao valor inteiro (tabela ASCII) dos 2 caracteres iniciais da marca e 2 caracteres iniciais do modelo do produto. Conforme explicado anteriormente.
void listarProdutos(**struct** produtos produto[10]);	Função sem retorno e que passa uma struct com os produtos cadastrados a fim de exibir na tela a lista de todos os produtos cadastrados. Deverá ser chamada quando solicitado no menu inicial e também ao iniciar uma venda para facilitar a escolha dos produtos.
void efetuarVenda(**struct** clientes cliente[3], **struct** produtos produto[10], **struct** carrinhos carrinho);	Função sem retorno e que passa a struct com os clientes, a struct com os produtos e a struct que é o carrinho de compras. Nesta função deverá ocorrer todos os passos para a realização da venda. Deverá ser chamada quando solicitado no menu inicial.
struct clientes localizaCliente(**int** codigo, **struct** clientes cliente[3]);	Função que passa o código do cliente e a lista de clientes com o objetivo de localizar nesta lista o cliente correspondente ao código passado. Ao término da busca, a função uma struct com o cliente encontrado para ser inserido no carrinho de compras. Deverá ser chamada no início da venda.
struct produtos localizaProduto(**int** codigo, **struct** produtos produto[10]);	Função que passa o código do produto e a lista de produtos com o objetivo de localizar nesta lista o produto correspondente ao código passado. Ao término da busca, a função uma struct com o produto encontrado para ser inserido no carrinho de compras. Deverá ser chamada quando escolhido o item da compra.
void finalizarVenda(**struct** carrinhos carrinho);	Função sem retorno e que passa a struct com o carrinho de compras. Esta função deverá finalizar a compra listando o carrinho e calculando os valores finais da compra (subtotal, desconto e total). Deverá ser chamada após terminar a inserção dos produtos
void listarCarrinho(**struct** carrinhos carrinho);	Função sem retorno e que passa a struct com o carrinho de compras. Esta função deverá listar o resumo da compra contendo os dados do cliente que está comprando e os produtos selecionados na compra. Deverá ser chamada pela função finalizarVenda.
double calculaSubtotal(**struct** carrinhos carrinho);	Função que passa uma struct com o carrinho de compras e retorna o valor do total comprado, ou seja, a soma de (qnt * valor do produto). Deverá ser chamada ao finalizar a compra.
double calculaTotal(**double** subtotal, **double** desconto);	Função que passa o subtotal da compra e o desconto a ser aplicado e retorna o valor final da compra. Exemplo: Subtotal 1500 e desconto de 10 (dez por cento) retorna o valor 1350. Deverá ser chamada ao finalizar a compra.

ALGORITMOS DE PROGRAMAÇÃO

Prof. Me. João Víctor Ramos

7) **O diagrama abaixo mostra um fluxo básico** de processos que auxilia a equipe no desenvolvimento. A equipe deve seguir o fluxo básico, fazendo adaptações caso implemente uma nova funcionalidade adicionais as apresentadas neste trabalho.



8) Todos os integrantes da equipe **<u>deverão obrigatoriamente fazer acesso ao portal do aluno</u>** e se inscreverem nos grupos disponíveis no portal.

9) As instruções listadas ao término deste documento **<u>deverão ser seguidas à risca</u>**.

10) A equipe deverá utilizar apenas comandos que foram ensinados na sala de aula. O professor poderá questionar qualquer aluno da equipe (mesmo não sendo ele responsável por esta parte) quanto a como cada parte do trabalho foi desenvolvido. Todos devem saber, caso contrário **todos poderão perder nota**.

ALGORITMOS DE PROGRAMAÇÃO

Prof. Me. João Víctor Ramos

ENTREGA DA ATIVIDADE

- Os exercícios devem ser feitos em formato de Texto utilizando o *Bloco de Notas* e deverão ser salvos com a extensão TXT. O nome do arquivo deverá ser seu nome completo conforme exemplo: GRUPO_10_MERCADODIGITAL.TXT
- O arquivo salvo deverá conter também **o nome completo dos alunos comentado no início do arquivo**.
- O trabalho deverá ser feito em equipes de **no mínimo 4 e no máximo 4 alunos**. Equipes maiores ou menores apenas com autorização do professor.
- Estes exercícios deverão ser entregues até a data estipulada no portal do aluno. Lembrando que o prazo estipulado é o prazo máximo, podendo ser entregue ANTES deste prazo caso o aluno preferir.
- Não serão aceitos trabalhos enviados DE OUTRA MANEIRA ou FORA DO PRAZO MÁXIMO. Os alunos que cometerem PLÁGIO (cópia de exercícios de outros alunos) será atribuído nota ZERO. Não serão aceitos trabalhos FORA DAS INSTRUÇÕES citadas.

REFERÊNCIAS

- AGUILAR, Luis Joyanes(Aut.). **Fundamentos de programação: algoritmos, estruturas de dados e objetos**. São Paulo: McGraw Hill do Brasil, 2008. 690 p.
- EDELWEISS, Nina, CASTRO, Maria A**. Algoritmos e Programação Com Exemplos Em Pascal e C**. Porto Alegre : Bookman, 2014.
- SOFFNER, Renato(Aut.). **Algoritmos e programação em linguagem C.** São Paulo: Saraiva, 2013. 196 p.
- ASCENCIO, Ana Fernanda Gomes;CAMPOS, Edilene Aparecida Veneruchi de(Auts.). **Fundamentos da programação de computadores: algoritmos - Pascal - C/C++(padrão ansi) e java.** 3. ed. São Paulo: Person Prentice Hall, 2012. 569 p.
- CORMEN, Thomas H.... [et al.] **Algoritmos: teoria e prática.** Tradução Vandenberg D. Souza. Rio de Janeiro: Ed. Campus, 2002.
- EDELWEISS, Nina, GALANTE Renata. **Estruturas de Dados.** Porto Alegre : Bookman, 2009.
- FORBELLONE, André L. V.; Henri Frederico E. **Lógica de programação: a construção de algoritmos e estrutura de dados**. 3. ed. São Paulo: Ed. Pearson Prentice Hall, 2005.
- GOODRICH, Michael T.;TAMASSIA, Roberto(Auts.);COPSTEIN, Bernardo. **Estruturas de dados & algoritmos em JAVA**. 5. ed. Porto Alegre: Bookman, 2013. 713 p.
- MANZANO, José Augusto Navarro Garcia; OLIVEIRA, Jayr Figueiredo de. **Algoritmos : lógica para desenvolvimento de programação de computadores.** 22. ed. São Paulo: Ed. Érica, 2009.